RUN: OOP Unit 3, Inheritance and Virtual Functions

Nov 5, 2026 · @shahan

All of Unit 3, with every Sessional-2 and endsem inheritance question solved where it belongs. Allow about 2 hours. **Every program here was compiled and run with g++ 16.**

How this run works

Read the idea, write the PYQ yourself on paper, then check the answer right under it. Each question sits in a box, and the line you'd circle in the exam is in bold.

What the paper likes: a multilevel hierarchy with constructor chaining (every endsem), virtual base class (three endsems running), and "base pointer, derived object, which function runs?" Learn those three and Unit 3 is yours.

Videos: in the CppNuts playlist, watch Inheritance, Types of Inheritance, Virtual Base Class (Diamond Problem), Virtual Function, Pure Virtual Function and Virtual Destructor. Slides are in `OOP/Notes/Unit 3 slides.pdf`.

Inheritance is just a family tree where the kids get the money and the furniture, but not the private diary.

Stop 1: Derived classes and visibility modes

A derived class takes the members of a base class and adds its own. You write it once and reuse it forever.

```
class Derived : visibility_mode Base {
    // new members
};
```

The visibility mode decides what the inherited members turn into inside the derived class. Private members of the base are never accessible in the derived class, whatever the mode.

Member in Base	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	not accessible	not accessible	not accessible

The default is **private** for `class` and **public** for `struct`. Forget the word `public` and your whole interface goes dark.

Advantages of inheritance: code reuse, faster development, easier maintenance and extension, and a natural model of "is-a" relationships.

> **PYQ, Endsem 2023 Q3(a)(I), 3 marks.** What are access specifiers in C++? How are they used to control access to class members? What is their role in inheritance?

Answer: Access specifiers decide who can touch a member.

Specifier	Inside the class	In a derived class	Outside, through an object
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No	No

In inheritance they do two jobs. The specifier on a member decides whether the derived class can see it at all. The visibility mode in `class D : mode B` then decides how visible the inherited members are in D, as in the table above.

```
#include <iostream>
using namespace std;

class Base {
public:    int a = 1;
protected: int b = 2;
private:   int c = 3;
};

class PubD : public Base {       // a stays public, b stays protected
public:
    void show() { cout << a << " " << b << endl; }   // c is not accessible
};
class ProD : protected Base {    // a and b become protected
public:
    void show() { cout << a << " " << b << endl; }
};
class PriD : private Base {      // a and b become private
public:
    void show() { cout << a << " " << b << endl; }
};

int main() {
    PubD x; ProD y; PriD z;
    cout << x.a << endl;   // fine: still public
    // cout << y.a;        // error: protected in ProD
    // cout << z.a;        // error: private in PriD
    x.show(); y.show(); z.show();
    return 0;
}
```

Output: `1`, then `1 2` three times.

Key line: protected is "private, but the kids can see it".

> **PYQ, Endsem 2023 Q3(c)(III), 2 marks.** If Base has a protected static data member data1, is it inherited into Derv? How can we access it within func1 of Derv?

Answer: A static member belongs to the class, not to an object, so there is only one copy. Derv doesn't get its own copy, but since data1 is protected, Derv can use it directly by name or as `Base::data1`. The program is in the next PYQ.

Yes, it's accessible in Derv, as `data1` or `Base::data1`, and both refer to the same single copy.

> **PYQ, Endsem 2023 Q3(c)(II), 2 marks.** Base has a public member function func1, and the publicly derived class Derv also has func1. How can we access func1 of Base within func1 of Derv?

Answer: Derv's func1 hides Base's. Use the scope resolution operator to reach past it.

```
#include <iostream>
using namespace std;

class Base {
protected:
    static int data1;
public:
    void func1() { cout << "Base::func1\n"; }
};
int Base::data1 = 10;

class Derv : public Base {
public:
    void func1() {
        Base::func1();                      // the base version
        cout << "Derv::func1, data1 = " << data1 << ", " << Base::data1 << endl;
    }
};

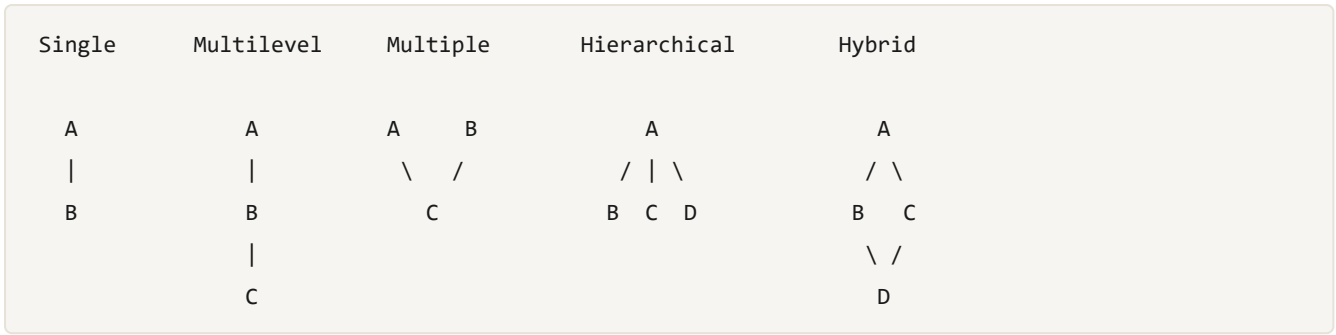
int main() {
    Derv d;
    d.func1();
    return 0;
}
```

Output: `Base::func1`, then `Derv::func1, data1 = 10, 10`.

Write `Base::func1();` inside Derv::func1. A plain `func1();` would call itself forever, which is recursion nobody asked for.

Stop 2: Types of inheritance

Five shapes. Draw the arrows from child to parent, which is how every textbook does it, and how the examiner expects to see it.



Type	Shape	Syntax
Single	One base, one derived	`class B : public A`
Multilevel	A chain: grandparent, parent, child	`class B : public A`, `class C : public B`
Multiple	One derived, two or more bases	`class C : public A, public B`
Hierarchical	One base, many derived	`class B : public A`, `class C : public A`
Hybrid	A mix, usually the diamond	Multiple plus hierarchical

The hybrid diamond is where the trouble starts, and Stop 4 deals with it.

> **Quick check.** Which type is `class Result : public Test, public Sports`, where both Test and Sports derive from Student?

Hybrid. Result alone is multiple inheritance, and Test and Sports from Student is hierarchical. Together they make the diamond.

Stop 3: Constructors and destructors in derived classes

When you create a derived object, the base part is built first, then the derived part. Destruction runs in reverse. It's like building a house: foundation first, and when you demolish it, the roof comes off first.

- **Construction order:** base, then derived. In multiple inheritance, bases are built in the order they're listed in the class header, not the initializer list.
- **Destruction order:** exactly reversed.
- **Passing arguments up:** if the base constructor takes arguments, the derived constructor must pass them through its initializer list.

```
Derived(int x, int y) : Base(x), d(y) { }
```

Virtual base classes are built before everything else, by the most derived class.

> **PYQ, Sessional-2 2021 Q2, 5 marks.** Write a C++ program showing how parameters pass from a derived class constructor to the base class constructor. Also display the order of execution of constructors. Model a real-world scenario with more than 4 classes.

Answer: A college staff chain of four levels: Person, Employee, Teacher, HOD. Each constructor hands its share of the arguments up to its parent.

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
public:
    Person(string n) : name(n) { cout << "1. Person constructor\n"; }
    ~Person() { cout << "Person destructor\n"; }
};

class Employee : public Person {
protected:
    int empId;
public:
    Employee(string n, int id) : Person(n), empId(id) { cout << "2. Employee constructor\n"; }
    ~Employee() { cout << "Employee destructor\n"; }
};

class Teacher : public Employee {
protected:
    string subject;
public:
    Teacher(string n, int id, string s) : Employee(n, id), subject(s) { cout << "3. Teacher
constructor\n"; }
    ~Teacher() { cout << "Teacher destructor\n"; }
};

class HOD : public Teacher {
    string dept;
public:
    HOD(string n, int id, string s, string d) : Teacher(n, id, s), dept(d) {
        cout << "4. HOD constructor\n";
    }
    ~HOD() { cout << "HOD destructor\n"; }
    void show() {
        cout << name << ", " << empId << ", " << subject << ", HOD of " << dept << endl;
    }
};

int main() {
    HOD h("Dr. Rao", 101, "Networks", "Computer Engg");
    h.show();
    return 0;
}
```

Output:

```
1. Person constructor
2. Employee constructor
3. Teacher constructor
4. HOD constructor
Dr. Rao, 101, Networks, HOD of Computer Engg
HOD destructor
Teacher destructor
Employee destructor
Person destructor
```

Constructors run top of the chain down (Person to HOD), destructors run bottom up (HOD to Person). The question says more than 4 classes, so in the exam add a fifth level, such as `Dean : public HOD`, the same way.

> **PYQ, Endsem 2025 Q3(b), 4.5 marks.** Use multilevel inheritance for hotel guests. Base class Guest has name and phone, with functions to input and display. Boarder derives from Guest and adds roomNo and tariff. LongStay derives from Boarder and adds contractNo and discountPercent. Implement default and parameterized constructors with proper constructor chaining.

Answer: Each level's `input()` and `display()` call the parent's version first, then handle their own members. Each constructor chains to the parent through the initializer list.

```
#include <iostream>
#include <string>
using namespace std;

class Guest {
protected:
    string name, phone;
public:
    Guest() : name("Unknown"), phone("-") {}
    Guest(string n, string p) : name(n), phone(p) {}
    void input() { cout << "Name and phone: "; cin >> name >> phone; }
    void display() { cout << "Guest: " << name << ", Phone: " << phone << endl; }
};

class Boarder : public Guest {
protected:
    int roomNo;
    double tariff;
public:
    Boarder() : Guest(), roomNo(0), tariff(0) {}
    Boarder(string n, string p, int r, double t) : Guest(n, p), roomNo(r), tariff(t) {}
    void input() {
        Guest::input();
        cout << "Room number and tariff: "; cin >> roomNo >> tariff;
    }
    void display() {
        Guest::display();
        cout << "Room: " << roomNo << ", Tariff per night: " << tariff << endl;
    }
};

class LongStay : public Boarder {
    int contractNo;
    double discountPercent;
public:
    LongStay() : Boarder(), contractNo(0), discountPercent(0) {}
    LongStay(string n, string p, int r, double t, int c, double d)
        : Boarder(n, p, r, t), contractNo(c), discountPercent(d) {}
    void input() {
        Boarder::input();
        cout << "Contract number and discount %: "; cin >> contractNo >> discountPercent;
    }
    void display() {
        Boarder::display();
        cout << "Contract: " << contractNo << ", Discount: " << discountPercent
             << "%, Effective tariff: " << tariff * (1 - discountPercent / 100) << endl;
```

```
    }
};

int main() {
    LongStay g1("Aman", "9876543210", 204, 3000, 77, 20);   // parameterized chain
    g1.display();
    LongStay g2;                                             // default chain
    g2.display();
    return 0;
}
```

Output for g1: `Guest: Aman, Phone: 9876543210`, `Room: 204, Tariff per night: 3000`, `Contract: 77, Discount: 20%, Effective tariff: 2400`. g2 prints the defaults.

The chaining is `LongStay(...) : Boarder(n, p, r, t)` **and** `Boarder(...) : Guest(n, p)`**.** That one line in each class is what the question means by proper constructor chaining.

> **PYQ, Endsem 2021 Q3(a), 6 marks.** A bank keeps depositors and borrowers. Base class Customer (name, phone-number). Derive Depositor (accno, balance) from Customer. Derive Borrower (loan-no, loan-amt) from Depositor. Write member functions to read and display the details of n customers, with relevant constructors.

Answer: The same multilevel pattern as the hotel, with an array of n Borrower objects, since a Borrower holds all three levels.

```
#include <iostream>
#include <string>
using namespace std;

class Customer {
protected:
    string name, phone;
public:
    Customer(string n = "", string p = "") : name(n), phone(p) {}
    void read() { cout << "Name and phone: "; cin >> name >> phone; }
    void display() { cout << "Name: " << name << ", Phone: " << phone << endl; }
};

class Depositor : public Customer {
protected:
    int accNo;
    double balance;
public:
    Depositor(string n = "", string p = "", int a = 0, double b = 0)
        : Customer(n, p), accNo(a), balance(b) {}
    void read() {
        Customer::read();
        cout << "Account number and balance: "; cin >> accNo >> balance;
    }
    void display() {
        Customer::display();
        cout << "Account: " << accNo << ", Balance: " << balance << endl;
    }
};

class Borrower : public Depositor {
    int loanNo;
    double loanAmt;
public:
    Borrower(string n = "", string p = "", int a = 0, double b = 0, int l = 0, double amt = 0)
        : Depositor(n, p, a, b), loanNo(l), loanAmt(amt) {}
    void read() {
        Depositor::read();
        cout << "Loan number and amount: "; cin >> loanNo >> loanAmt;
    }
    void display() {
        Depositor::display();
        cout << "Loan: " << loanNo << ", Amount: " << loanAmt << endl;
    }
};
```

```
int main() {
    int n;
    cout << "How many customers? "; cin >> n;
    Borrower* list = new Borrower[n];
    for (int i = 0; i < n; i++) list[i].read();
    for (int i = 0; i < n; i++) { cout << "\nCustomer " << i + 1 << endl; list[i].display(); }
    delete[] list;
    return 0;
}
```

Tested with one customer, input `Ali 999 5 2000 77 50000`, which prints all three levels: name and phone, account 5 with balance 2000, loan 77 for 50000.

Default arguments on every constructor let `new Borrower[n]` work, since an array needs a default constructor.

Stop 4: Ambiguity and virtual base classes

Ambiguity type 1, same name in two bases. If A and B both have `show()` and `class C : public A, public B`, then `c.show()` won't compile, because the compiler can't pick. Fix it with scope resolution, `c.A::show()`, or override `show()` in C.

Ambiguity type 2, the diamond. Test and Sports both inherit Student, and Result inherits both. Result now holds **two copies** of Student's members, so `roll` is ambiguous. That's two birth certificates for one kid.

Fix: make the middle classes inherit **virtually**. Then only one shared copy of the base is kept, and the most derived class constructs it.

```
          Student
         /       \
  virtual         virtual
     Test         Sports
         \       /
          Result      <- one Student inside, not two
```

> **PYQ, Endsem 2025 Q3(a), 4.5 marks; Endsem 2022 Q3(b), 6 marks; Endsem 2021 Q3(c), 6 marks.** When does ambiguity arise in multiple inheritance, and how can you resolve it? What are virtual base classes and why do they matter? Demonstrate the concept of a virtual base class with a suitable program.

Answer: Ambiguity arises in the two cases above: the same member name in two bases, or a common grandparent reached by two paths. The first is fixed with `::`. The second is fixed with a virtual base class, which keeps a single shared copy of the grandparent, saves memory, and removes the ambiguity.

```
#include <iostream>
using namespace std;

class Student {
protected:
    int roll;
public:
    Student() : roll(0) { cout << "Student constructor\n"; }
    void setRoll(int r) { roll = r; }
};

class Test : virtual public Student {       // virtual here
protected:
    int marks;
public:
    void setMarks(int m) { marks = m; }
};

class Sports : virtual public Student {     // and here
protected:
    int score;
public:
    void setScore(int s) { score = s; }
};

class Result : public Test, public Sports {
public:
    void show() {
        cout << "Roll " << roll << ", total " << marks + score << endl;  // no ambiguity
    }
};

int main() {
    Result r;               // "Student constructor" prints once
    r.setRoll(42);
    r.setMarks(80);
    r.setScore(15);
    r.show();
    return 0;
}
```

Output: `Student constructor` once, then `Roll 42, total 95`.

Without the two `virtual` keywords, `roll` in Result is ambiguous and the program does not compile. Say that line in the exam, it's what the marks are for.

Stop 5: Base pointers and virtual functions

A base class pointer can point to a derived object, because a Dog **is an** Animal. The catch: by default, the call is chosen by the **pointer's type**, so you get the base version. Marking the function `virtual` makes the call follow the **object's type** instead.

```
class Animal { public: virtual void sound() { cout << "..."; } };
class Dog : public Animal { public: void sound() { cout << "Woof"; } };

Animal* p = new Dog;
p->sound();     // Woof with virtual, ... without it
```

How it works: each class with virtual functions gets a **vtable**, a table of function addresses, and each object carries a hidden pointer to its class's vtable. The call is looked up there at run time.

> **PYQ, Sessional-2 2022 Q3(b), 1 mark.** Explain pointers to derived class objects.

Answer: A base class pointer can hold the address of a derived object, as in `Base* p = &d;`. Through it you can reach only the members declared in Base. Non-virtual calls use Base's versions, while virtual calls use the derived object's versions. The reverse isn't allowed without a cast: a `Derived*` can't point to a plain Base object.

Base pointer, derived object: allowed, and it's the foundation of run-time polymorphism.

> **PYQ, Sessional-2 2023 Q1(a), 1 mark.** Inheritance enables run-time polymorphism. Justify this statement.

Answer: Run-time polymorphism needs one interface with many implementations, chosen while the program runs. Inheritance provides both halves. The base class defines the common interface as virtual functions, and each derived class overrides them. Since a base pointer can point to any derived object, the same call `p->f()` runs a different function depending on the object.

No inheritance means no base pointer to a derived object, and so no run-time polymorphism.

> **PYQ, Endsem 2023 Q3(c)(I), 2 marks.** Differentiate between late (dynamic) binding and early (static) binding, giving at least two differences.

Answer:

Basis	Early (static) binding	Late (dynamic) binding
When the call is resolved	At compile time	At run time
Decided by	Pointer or reference type	Actual object type
Used for	Normal functions, overloading	Virtual functions
Speed	Faster, a direct call	Slightly slower, a vtable lookup
Flexibility	Less	More, new derived classes work without changing old code

> **PYQ, Endsem 2021 Q3(b), 6 marks.** Differentiate between method overriding and method overloading with a suitable program.

Answer:

Basis	Overloading	Overriding
Where	Same class	Base and derived class
Signature	Must differ in parameters	Must be the same
Binding	Compile time	Run time, when virtual
Inheritance needed	No	Yes

```
#include <iostream>
using namespace std;

class Calc {
public:
    int add(int a, int b) { return a + b; }               // overloading:
    double add(double a, double b) { return a + b; }      // same name, different parameters
    virtual void show() { cout << "Calc\n"; }
    virtual ~Calc() {}
};

class SciCalc : public Calc {
public:
    void show() override { cout << "SciCalc\n"; }         // overriding: same signature
};

int main() {
    Calc c;
    cout << c.add(2, 3) << " " << c.add(2.5, 3.5) << endl;
    Calc* p = new SciCalc;
    p->show();
    delete p;
    return 0;
}
```

Output: `5 6`, then `SciCalc`.

Overloading means same name, different parameters, in the same class. Overriding means same signature, redefined in the derived class.

> **PYQ, Sessional-2 2021 Q1, 5 marks.** Write a C++ program that uses virtual functions to achieve run-time polymorphism. Model a real-world scenario.

Answer: A payment system. The checkout code holds only `Payment*` pointers and never needs to know which kind it has.

```
#include <iostream>
using namespace std;

class Payment {
protected:
    double amount;
public:
    Payment(double a) : amount(a) {}
    virtual void pay() { cout << "Generic payment of " << amount << endl; }
    virtual ~Payment() {}
};

class UPI : public Payment {
public:
    UPI(double a) : Payment(a) {}
    void pay() override { cout << "Paid " << amount << " by UPI, no fee" << endl; }
};

class Card : public Payment {
public:
    Card(double a) : Payment(a) {}
    void pay() override { cout << "Paid " << amount * 1.02 << " by card, with 2% fee" << endl; }
};

int main() {
    Payment* p[3] = { new UPI(500), new Card(1000), new UPI(250) };
    for (int i = 0; i < 3; i++) p[i]->pay();       // which pay()? decided at run time
    for (int i = 0; i < 3; i++) delete p[i];
    return 0;
}
```

Output: `Paid 500 by UPI, no fee`, `Paid 1020 by card, with 2% fee`, `Paid 250 by UPI, no fee`.

One loop, one call, `p[i]->pay()`, and three different behaviours. That's run-time polymorphism.

Stop 6: Pure virtual functions and abstract classes

A **pure virtual function** has no body in the base class, just `= 0`:

```
virtual double area() = 0;
```

It says "every child must define this, and I have no sensible default". A class with at least one pure virtual function is an **abstract class**.

- You **can't create objects** of an abstract class.
- You **can** have pointers and references to it, and that's the whole point.
- A derived class that doesn't override every pure virtual function is abstract too.

- An abstract class can still have data members, constructors and normal functions.

Think of it as the job description. Nobody hires the job description, they hire a person who fits it.

> **PYQ, Sessional-2 2022 Q3(a), 1 mark.** Explain pure virtual functions with an example.

Answer: A pure virtual function is a virtual function declared with `= 0` and no body in the base class. It forces every concrete derived class to provide its own definition, and makes the base class abstract.

```
class Shape { public: virtual double area() = 0; };
class Circle : public Shape {
    double r;
public:
    Circle(double x) : r(x) {}
    double area() { return 3.14 * r * r; }
};
```

`= 0` means no body here, so the derived class must define it, and Shape can't be instantiated.

> **PYQ, Endsem 2023 Q3(a)(II), 3 marks.** Explain abstract classes, virtual functions and pure virtual functions in C++. How do these enable run-time polymorphism? Explain with a simple example.

Answer:

Term	Meaning
Virtual function	A base function marked `virtual`, so a call through a base pointer runs the derived version
Pure virtual function	A virtual function with `= 0` and no body, which must be overridden
Abstract class	A class with at least one pure virtual function, with no objects allowed

The abstract class fixes a common interface. Each derived class overrides it. Code written against `Base*` then calls the right version for whatever object it receives, decided at run time through the vtable. The Shape program in the next PYQ is the example.

> **PYQ, Endsem 2023 Q3(b), 6 marks.** (I) Define an abstract class Shape with a protected attribute name, a public member function print, a public pure virtual function area, and a 1-argument constructor taking the name. Define a publicly derived class Rectangle with length and width and a 3-argument constructor, passing the name to Shape through an initializer list. Override print and area. (II) Define class Square, publicly derived from Rectangle, with a 2-argument constructor taking side and name, passing them to Rectangle in the initializer list. Override print and area. What is the output of the given code?

Answer:

```
#include <iostream>
#include <string>
using namespace std;

class Shape {
protected:
    string name;
public:
    Shape(string n) : name(n) {}
    virtual void print() { cout << "Shape: " << name << endl; }
    virtual double area() = 0;
    virtual ~Shape() {}
};

class Rectangle : public Shape {
protected:
    double length, width;
public:
    Rectangle(double l, double w, string n) : Shape(n), length(l), width(w) {}
    void print() override {
        cout << name << " with length " << length << " and width " << width << endl;
    }
    double area() override {
        cout << "Area of " << name << " = " << length * width << endl;
        return length * width;
    }
};

class Square : public Rectangle {
    double side;
public:
    Square(double s, string n) : Rectangle(s, s, n), side(s) {}
    void print() override { cout << name << " with side " << side << endl; }
    double area() override {
        cout << "Area of " << name << " = " << side * side << endl;
        return side * side;
    }
};

int main() {
    Shape* myShape = new Rectangle(4, 2, "Rectangle");
    Rectangle* myRectangle = new Square(5, "Square");
    myShape->area();
    myShape->print();
    myRectangle->area();
    myRectangle->print();
```

```
    delete myShape;
    delete myRectangle;
    return 0;
}
```

Output:

```
Area of Rectangle = 8
Rectangle with length 4 and width 2
Area of Square = 25
Square with side 5
```

Every call runs the version of the actual object, because print and area are virtual. The trap: if you leave `print` non-virtual in Shape, as the question literally words it, then `myShape->print()` prints `Shape: Rectangle` instead. Write that one line as a note and you've covered both readings.

> **PYQ, Sessional-2 2023 Q1(b), 1 mark.** Define a class Element with no data members, two member functions `void print()` and `double norm()`, a default constructor and a default destructor. print should be virtual and norm pure virtual. Can you instantiate an object of class Element? Why or why not?

Answer:

```
class Element {
public:
    Element() {}
    virtual ~Element() {}
    virtual void print() { cout << "Element" << endl; }
    virtual double norm() = 0;          // pure virtual
};
```

No, Element can't be instantiated, because norm() is pure virtual, which makes Element abstract. `Element e;` is a compile error. `Element* p;` is fine.

> **PYQ, Sessional-2 2023 Q1(c), 3 marks.** Define class PointND with a single data member, a pointer to double. PointND derives publicly from Element. Provide a default constructor, a parameterized constructor that takes a double array and its size aSize (int) and copies the data, and a default destructor.

Answer: The question says a single data member, but the size has to be remembered somewhere for print and norm to work, so state that assumption and add `size`.

```
#include <iostream>
#include <cmath>
using namespace std;

// Element as above

class PointND : public Element {
    double* data;
    int size;
public:
    PointND() : data(nullptr), size(0) {}
    PointND(const double arr[], int aSize) : size(aSize) {
        data = new double[size];
        for (int i = 0; i < size; i++) data[i] = arr[i];
    }
    ~PointND() { delete[] data; }
    void print() override {
        cout << "(";
        for (int i = 0; i < size; i++) cout << data[i] << (i < size - 1 ? ", " : "");
        cout << ")" << endl;
    }
    double norm() override {
        double s = 0;
        for (int i = 0; i < size; i++) s += data[i] * data[i];
        return sqrt(s);
    }
};

int main() {
    double someData[3] = {1, 2, 3};
    Element* anElementPtr = new PointND(someData, 3);
    anElementPtr->print();
    cout << anElementPtr->norm() << endl;
    delete anElementPtr;
    return 0;
}
```

Output: `(1, 2, 3)`, then `3.74166`, which is the square root of 14.

PointND must override norm(), or it stays abstract too.

> **PYQ, Sessional-2 2023 Q2(a), 2.5 marks.** Will this code compile and run without problems in C++? What if similar code was written in Java? `double someData[3] = {1,2,3}; Element* anElementPtr = new PointND(someData,3); delete anElementPtr;`

Answer: In C++ it compiles, but whether it runs cleanly depends on Element's destructor. If Element's destructor is a plain default one, `delete anElementPtr` calls only `~Element()`. PointND's destructor never

runs, so the array behind `data` leaks, and formally the behaviour is undefined. Making `~Element()` virtual, and freeing the array with `delete[]` in `~PointND()`, fixes it. That's exactly what the code above does.

In Java the exact code won't compile, because Java has no pointers and no `delete`. The Java version is `Element e = new PointND(someData, 3);` and the garbage collector frees the object once nothing refers to it, so the leak can't happen.

Stop 7: Virtual destructors and the rules

If you `delete` a derived object through a base pointer and the base destructor isn't virtual, only the base destructor runs. Whatever the derived part allocated is never freed. A virtual destructor makes `delete` start at the real object's destructor and work up the chain.

Rule of thumb: if a class has any virtual function, give it a virtual destructor.

> **PYQ, Sessional-2 2022 Q3(c), 1 mark.** Explain the virtual destructor with an example.

Answer:

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual ~Base() { cout << "Base destructor\n"; }
};

class Derived : public Base {
    int* arr;
public:
    Derived() { arr = new int[100]; }
    ~Derived() { delete[] arr; cout << "Derived destructor\n"; }
};

int main() {
    Base* p = new Derived;
    delete p;      // Derived destructor, then Base destructor
    return 0;
}
```

Output: `Derived destructor`, then `Base destructor`.

Without `virtual`, only `Base destructor` prints and the 100 ints leak. Note that constructors can't be virtual, but destructors can and often should be.

> **PYQ, Endsem 2022 Q3(c)(i), 3 marks.** What are pure virtual functions? What rules must be kept in mind while deciding virtual functions?

Answer: Pure virtual functions are covered in Stop 6. The rules for virtual functions:

1. A virtual function must be a **member** of a class. It can't be static, and it can't be a friend.
2. It's accessed through a **pointer or reference** to the base class to get run-time behaviour. Called on an object directly, it binds statically.
3. The prototype in the base and derived class must be **identical**: same name, parameters and const-ness. Otherwise it's a new function that hides the old one, not an override.
4. It must be **defined** in the base class, unless it's declared pure with `= 0`.
5. **Constructors can't be virtual**, but destructors can, and should be in a polymorphic base.
6. Once virtual, it **stays virtual** in all derived classes, whether or not they repeat the keyword.
7. A base pointer can point to a derived object, but not the other way round.
8. If a derived class doesn't redefine it, the base version is used.

Stop 8: Full hierarchy programs

These are the long-answer versions: a base class with virtual functions, a few derived classes, and a loop over base pointers. Same skeleton every time, which is great news for you.

> **PYQ, Sessional-2 2022 Q2, 4 marks.** A bookshop sells books and video tapes, both derived from a base class media, which has title and price. Book adds the number of pages, and tape adds the playing time. Each class has read() and show(), which must be virtual in the base class. Model the hierarchy and process the objects using pointers to the base class.

Answer:

```
#include <iostream>
#include <string>
using namespace std;

class Media {
protected:
    string title;
    float price;
public:
    virtual void read() { cout << "Title and price: "; cin >> title >> price; }
    virtual void show() { cout << "Title: " << title << ", Price: " << price << endl; }
    virtual ~Media() {}
};

class Book : public Media {
    int pages;
public:
    void read() override { Media::read(); cout << "Pages: "; cin >> pages; }
    void show() override { Media::show(); cout << "Pages: " << pages << endl; }
};

class Tape : public Media {
    float playTime;
public:
    void read() override { Media::read(); cout << "Playing time (min): "; cin >> playTime; }
    void show() override { Media::show(); cout << "Playing time: " << playTime << " min" << endl; }
};

int main() {
    Media* list[10];
    int n = 0, choice;
    do {
        cout << "\n1. Book  2. Tape  3. Show all  4. Exit: ";
        cin >> choice;
        if (choice == 1 && n < 10)      { list[n] = new Book; list[n++]->read(); }
        else if (choice == 2 && n < 10) { list[n] = new Tape; list[n++]->read(); }
        else if (choice == 3)
            for (int i = 0; i < n; i++) list[i]->show();   // base pointer, right version
    } while (choice != 4);
    for (int i = 0; i < n; i++) delete list[i];
    return 0;
}
```

Tested with a book (`Cpp 450 300`) and a tape (`Songs 120 60`). Show all printed each with its own extra field.

`list[i]->show()` **picks Book::show or Tape::show at run time, because show is virtual in Media.**

> **PYQ, Endsem 2022 Q3(c)(ii), 3 marks.** Create a base class shape that stores two double values used to compute areas. Derive triangle and rectangle from it. Add get_data() to the base class to set the data, and a virtual display_area() that each derived class redefines.

Answer:

```
#include <iostream>
using namespace std;

class shape {
protected:
    double x, y;
public:
    void get_data(double a, double b) { x = a; y = b; }
    virtual void display_area() { cout << "No area for a generic shape" << endl; }
    virtual ~shape() {}
};

class triangle : public shape {
public:
    void display_area() override { cout << "Triangle area = " << 0.5 * x * y << endl; }
};

class rectangle : public shape {
public:
    void display_area() override { cout << "Rectangle area = " << x * y << endl; }
};

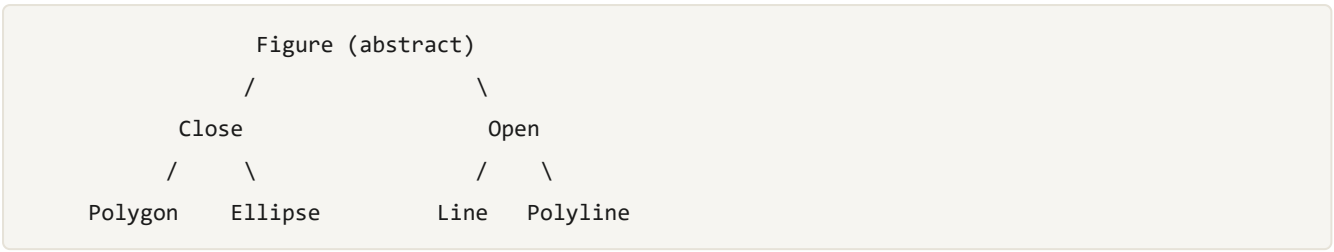
int main() {
    shape* p;
    triangle t;
    rectangle r;
    p = &t; p->get_data(10, 5); p->display_area();
    p = &r; p->get_data(10, 5); p->display_area();
    return 0;
}
```

Output: `Triangle area = 25`, then `Rectangle area = 50`.

For a triangle x is the base and y the height, so its area is half of x times y.

> **PYQ, Endsem 2022 Q3(a), 6 marks.** Write a C++ program for a graphics class hierarchy. Create an abstract base class Figure, and derive Close and Open from it. Derive Polygon and Ellipse from Close, and Line and Polyline from Open. Define three objects (triangle, rectangle, pentagon) of class Polygon. All classes need appropriate member functions, including constructors and destructors.

Answer:


Figure (abstract)
Close
Open
Polygon
Ellipse
Line
Polyline

```
#include <iostream>
using namespace std;

class Figure {                              // abstract base
public:
    Figure() { cout << "Figure created\n"; }
    virtual void draw() = 0;
    virtual ~Figure() { cout << "Figure destroyed\n"; }
};

class Close : public Figure {               // closed shapes
public:
    Close() { cout << "  Close created\n"; }
    ~Close() { cout << "  Close destroyed\n"; }
};

class Open : public Figure {                // open shapes
public:
    Open() { cout << "  Open created\n"; }
    ~Open() { cout << "  Open destroyed\n"; }
};

class Polygon : public Close {
    int sides;
public:
    Polygon(int s) : sides(s) { cout << "    Polygon with " << s << " sides created\n"; }
    void draw() override { cout << "Drawing a polygon with " << sides << " sides\n"; }
    ~Polygon() { cout << "    Polygon destroyed\n"; }
};

class Ellipse : public Close {
public:
    void draw() override { cout << "Drawing an ellipse\n"; }
};

class Line : public Open {
public:
    void draw() override { cout << "Drawing a line\n"; }
};

class Polyline : public Open {
public:
    void draw() override { cout << "Drawing a polyline\n"; }
};

int main() {
```

```
    Polygon triangle(3), rectangle(4), pentagon(5);
    Figure* f[3] = { &triangle, &rectangle, &pentagon };
    for (int i = 0; i < 3; i++) f[i]->draw();
    return 0;
}
```

Output: for each Polygon, `Figure created`, `Close created`, `Polygon with n sides created`. Then the three draw lines, for 3, 4 and 5 sides. At the end, each object is destroyed in reverse: Polygon, then Close, then Figure.

Close and Open don't override draw(), so they stay abstract too. Only the four leaf classes can have objects.

That's Unit 3. If you can write the virtual base class program and the hotel chain without looking, you've covered three endsems' worth of Q3. Top up the tea, and give the cat a well-earned scratch.